

Reviewer Pack — Minimal Number of Generators in Group Presentations and ACC^0 ...

Entry a314d41ad4e3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 13:40:52 UTC

Minimal Number of Generators in Group Presentations and ACC^0 Circuit Lower Bounds for XOR-AND Networks

Entry ID: a314d41ad4e3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 13:40:52 UTC

1. Conjecture

Field A (mathematical branch): Group Theory (Group Presentations) **Field B** (complexity object): Complexity Theory: ACC^0 Circuit Complexity

Statement:

[‘For all explicit Boolean functions f , the minimal number of generators g required in a group presentation of the syntactic monoid M_f is upper-bounded by $O(|f| \log n)$, where n is the size of the input for f . Furthermore, any ACC^0 circuit computing f with depth d must have at least 2^d generators.’, ‘For all explicit Boolean functions f , if there exists a group presentation of M_f requiring fewer than $O(|f| \log n)$ generators, then there exists an ACC^0 circuit computing f with depth less than $2^{\{|f|\}}$.’, ‘No Boolean function in P can be computed by an ACC^0 circuit of depth d unless there is a group presentation of its syntactic monoid with at most 2^d generators.’]

Rationale (proposer’s reasoning):

[‘Group presentations have been used to encode computational structures, but their connection to circuit complexity remains largely unexplored. This conjecture aims to bridge this gap by establishing a quantitative relation between the minimal number of generators in group presentations and ACC^0 circuit lower bounds.’, ‘The use of group theory in complexity theory could potentially uncover new insights into the structure of computation and lead to novel algorithms or proof techniques.’, ‘By relating ACC^0 complexity directly to group theory, we may be able to derive non-algebraic results about computational complexity.’]

Taxonomy category: ACC_SIPSER (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3975797c82f223d8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

A conjecture is supported if for all explicit Boolean functions f with input size $n \leq 40$, there exists a group presentation of M_f requiring at most $O(|f| \log n)$ generators and an ACC^0 circuit with depth $d \geq 2^{\{|f|\}}$ that computes f . The conjecture is falsified if any function f with input size $n \leq 40$ has a group presentation requiring fewer than $O(|f| \log n)$ generators and no ACC^0 circuit with depth less than $2^{\{|f|\}}$ can compute f .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "group theory" AND "syntactic monoid" AND "ACC⁰ complexity" - "group presentation" AND "Boolean function" AND "circuit lower bound" - "generators" IN GROUP PRESENTATION AND "ACC⁰ circuit depth"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def syntactic_monoid_generators(f):
        n = len(f)
        generators = []
        for i in range(n):
            for j in range(i+1, n):
                if f[i] == f[j]:
                    continue
                new_f = [f[k] ^ (f[i] & f[j]) for k in range(2**n)]
                if new_f not in generators:
                    generators.append(new_f)
        return len(generators)

    def acc0_circuit_depth(f):
        n = len(f)
```

```

depth = 1
while True:
    new_f = [f[k] ^ (f[i] & f[j]) for i, j in itertools.combinations(range(n), 2)]
    if new_f == f:
        return depth
    f = new_f
    depth += 1

n = random.randint(5, 40)
f = generate_boolean_function(n)

generators = syntactic_monoid_generators(f)
circuit_depth = acc0_circuit_depth(f)

metric_value = generators * math.log(n)
conjecture_holds = (generators <= metric_value) and (circuit_depth >= 2**n)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "Generators vs LogN",
    "metric_value": metric_value,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 1000003) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot confirm whether the conjecture holds for all explicit Boolean functions f with input size $n \leq 40$. | next: Run the test again to ensure it completes within the time limit and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18031
2	preregistration	ollama_remote	glm4:latest	0	0	10523
3	novelty	ollama_remote	glm4:latest	0	0	8569
4	novelty	ollama_remote	glm4:latest	0	0	9021
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14251
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9833
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12060
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7891
9	judge	ollama_remote	glm4:latest	0	0	13605

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 103783 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/a314d41ad4e3.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a314d41ad4e3.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/a314d41ad4e3.tar.gz` (if generated)